

Groq API – Home Assignment

API execution evidence for S01, S02 and S03 based on the provided Postman screenshots.

Scenario	Purpose	Request / Model	Execution Result
S01 – Text to Audio	Convert the supplied text into speech using Groq's audio endpoint.	POST /openai/v1/audio/speech Model: canopylabs/orpheus-v1-english Voice: diana Format: wav	200 OK Audio output generated and playable in Postman.
S02 – MultiLingual	Use the chat completion API to answer the user's Tamil request in the same language.	POST /openai/v1/chat/completions Model: llama-3.3-70b-versatile	200 OK Tamil response returned successfully.
S03 – Generate API Test Cases from Swagger Code	Generate executable Java Rest Assured/TestNG automation code from Swagger Petstore API requirements.	POST /openai/v1/chat/completions Model: openai/gpt-oss-120b	200 OK Java Rest Assured/TestNG code returned successfully.

Evidence: The complete screenshots supplied for each API call are included in the following pages without cropping the request or response areas.

S01 – Text to Audio | Complete Execution Proof

Postman request and successful 200 OK response with the generated WAV audio player.

The image displays the Postman application interface. At the top, the title bar reads "GenAI-Aug-HomeAssignment : S01-Text to Audio". The main area shows a POST request to the URL "https://api.groq.com/openai/v1/audio/speech". The request body is a JSON object with the following fields: "model": "canopylabs/orpheus-v1-english", "input": "[Cheerful] vanakkam. I am a language model created by Groq. I am here to help you with any questions you may have. How can I assist you today?", "voice": "diana", and "response_format": "wav". The response status is "200 OK" with a duration of "1.66 s" and a size of "402.04 KB". Below the response, there is an audio player with a play button, a progress bar showing "0:08 / 0:08", and a volume icon.

```
1 {
2   "model": "canopylabs/orpheus-v1-english",
3   "input": "[Cheerful] vanakkam. I am a language model created by Groq. I am here to help you with any questions you may have. How can I assist you today?",
4   "voice": "diana",
5   "response_format": "wav"
6 }
```

200 OK • 1.66 s • 402.04 KB

0:08 / 0:08

S02 – MultiLingual | Execution Proof 1 of 3

Complete screenshot supplied for the MultiLingual request, including request body and response.

The image shows a web browser window with a REST client interface. The top bar shows the URL "https://api.groq.com/openai/v1/chat/completions" and a "Send" button. Below the URL bar, there are tabs for "Docs", "Params", "Authorization", "Headers", "Body", "Scripts", and "Settings". The "Body" tab is selected, and it shows a JSON request body. The request body is a JSON object with a "model" field set to "llama-3.3-70b-versatile" and a "messages" array. The "messages" array contains two objects: one with "role": "system" and "content": "You are an API Specialist. Always respond in the same language as the user.", and another with "role": "user" and "content": "மிகவும் நன்றி: இந்த API-ஐ எப்படி பயன்படுத்துவது என்று சொல்லுங்கள்." The response is shown in the "JSON" tab, and it is a JSON object with a "id" field set to "1-86e2ed46-f868-4584-8557-e4de19b8e075", a "it.completion" field, a "36636722", a "na-3.3-70b-versatile", and a "role": "assistant" field. The "content" field contains a long Tamil text response. The response is also shown in the "Text" tab, and it is a long Tamil text response. The response is also shown in the "JSON" tab, and it is a JSON object with a "id" field set to "1-86e2ed46-f868-4584-8557-e4de19b8e075", a "it.completion" field, a "36636722", a "na-3.3-70b-versatile", and a "role": "assistant" field. The "content" field contains a long Tamil text response. The response is also shown in the "Text" tab, and it is a long Tamil text response.

S02 – MultiLingual | Execution Proof 2 of 3

Complete screenshot supplied for the MultiLingual request, including request body and response.

The screenshot displays a REST client interface with a POST request to `https://api.groq.com/openai/v1/chat/completions`. The request body is in raw JSON format, containing a system message and a user message in Tamil. The response is a 200 OK status with a JSON body containing chat completion details.

Request Body (Raw JSON):

```
1 {
2   "model": "llama-3.3-70b-versatile",
3   "messages": [
4     {
5       "role": "system",
6       "content": "You are an API Specialist. Always respond in the same language as the user."
7     },
8     {
9       "role": "user",
10      "content": "மிகவும் நன்றி! இந்த API-ஐ எப்படி பயன்படுத்துவது என்று சொல்லுங்கள்."
11    }
12  ]
13 }
```

Response Body (JSON):

```
1 {
2   "id": "chatcmpl-86e2ed46-f868-4684-8557-e4de19b8e075",
3   "object": "chat.completion",
4   "created": 1786636722,
5   "model": "llama-3.3-70b-versatile",
6   "choices": [
7     {
8       "index": 0,
9       "message": {
10        "role": "assistant",
11        "content": "நன்றி! இந்த API ஐ பயன்படுத்துவது எளிதானது. நீங்கள் முதலில் ஒரு கணக்கு உருவாக்க வேண்டும், பின்னர் உங்கள் API கியை பெற வேண்டும். அதன் பிறகு, நீங்கள் பயன்படுத்த தொடங்கலாம்.\n\nஇந்த API ஐ பயன்படுத்துவதற்கு, நீங்கள் ஒரு முகவர் அல்லது ஒரு நிரலாக்க மொழியைப் பயன்படுத்த வேண்டும். நீங்கள் ஒரு முகவரைப் பயன்படுத்துகிறீர்கள் என்றால், நீங்கள் இந்த API ஐ அணுக ஒரு கோரிக்கையை அனுப்ப வேண்டும்.\n\nநீங்கள் ஒரு நிரலாக்க மொழியைப் பயன்படுத்துகிறீர்கள் என்றால், நீங்கள் நிரலாக்க மொழி ஞாலத்தை பயன்படுத்த வேண்டும். நீங்கள் ஒரு நிரலாக்க மொழி ஞாலத்தை பயன்படுத்துகிறீர்கள் என்றால், நீங்கள் இந்த API ஐ அணுக ஒரு கோரிக்கையை \n\nஇந்த API ஐ பயன்படுத்துவதற்கு, நீங்கள் ஒரு API கியை தேவை. நீங்கள் உங்கள் API கியை பெறாவிட்டால், நீங்கள் இந்த API ஐ பயன்படுத்த தொடங்கலாம்.\n\nஇந்த API பயன்படுத்துவதற்கு, நீங்கள் ஒரு கோரிக்கையை அனுப்ப வேண்டும். கோரிக்கையை அனுப்புவதற்கு, நீங்கள் ஒரு முகவர் அல்லது ஒரு நிரலாக்க மொழியைப் பயன்படுத்த வேண்டும். நீங்கள் ஒரு கோரிக்கையை அனுப்ப வேண்டும். நீங்கள் உங்களுக்கு எந்த கேள்விகளையும் கேட்கலாம்."
12      },
13      "logprobs": null,
14      "finish_reason": "stop"
15    }
16  ],
17   "usage": {
18     "queue_time": 0.059266712,
19     "prompt_tokens": 133,
20     "prompt_time": 0.007154067,
21   }
22 }
```

S02 – MultiLingual | Execution Proof 3 of 3

Complete screenshot supplied for the MultiLingual request, including request body and response.

The screenshot displays a REST client interface for a POST request to `https://api.groq.com/openai/v1/chat/completions`. The request body is in raw JSON format, containing a system message in English and a user message in Tamil. The response is a 200 OK status with a JSON body detailing usage statistics and the completion text in Tamil.

Request Body (JSON):

```
1 {
2   "model": "llama-3.3-70b-versatile",
3   "messages": [
4     {
5       "role": "system",
6       "content": "You are an API Specialist. Always respond in the same language as the user."
7     },
8     {
9       "role": "user",
10      "content": "மிகவும் நன்றி! இந்த API-ஐ எப்படி பயன்படுத்துவது என்று சொல்லுங்கள்."
11    }
12  ]
13 }
```

Response Body (JSON):

```
12   },
13   "logprobs": null,
14   "finish_reason": "stop"
15 },
16 ],
17 "usage": {
18   "queue_time": 0.059266712,
19   "prompt_tokens": 133,
20   "prompt_time": 0.007154067,
21   "completion_tokens": 1621,
22   "completion_time": 3.385618865,
23   "total_tokens": 1654,
24   "total_time": 3.392772922
25 },
26 "usage_breakdown": null,
27 "system_fingerprint": "fp_dae98b5ecb",
28 "x_groq": {
29   "id": "req_01kzxxhm]hepyvc0qn7zc9sevy",
30   "seed": 1954654365
31 },
32 "service_tier": "on_demand"
33 }
```

Response Text (Tamil):

\n\nஇந்த API ஐ பயன்படுத்துவதற்கு, நீங்கள் ஒரு API கியை தேவை. நீங்கள் உங்கள் API கியை பெறுவீட்டால், நீங்கள் இந்த API ஐ பயன்படுத்த தொடங்கலாம்.\n\nஇந்த API பயன்படுத்துவதற்கு, நீங்கள் ஒரு கோரிக்கையை அனுப்ப வேண்டும். கோரிக்கையை அனுப்புவதற்கு, நீங்கள் ஒரு முகவர அல்லது ஒரு நிரலாக்க மொழியைப் பயன்படுத்த வேண்டி API ஐ பயன்படுத்துவதற்கு, நீங்கள் ஒரு API கியை தேவை. நீங்கள் உங்கள் API கியை பெறுவீட்டால், நீங்கள் இந்த API ஐ பயன்படுத்த தொடங்கலாம்.\n\nநீங்கள் இந்த API ஐ பயன்படுத்துவதற்கு உதவுவதில் மகிழ்ச்சி அடைகிறோம். நீங்கள் எங்களுக்கு எந்த கேள்விகளையும் கேட்கலாம்."

S03 – Generate API Test Cases from Swagger Code | Execution Proof

1 of 2

Complete screenshot supplied for the Swagger-to-test-code generation request, including request and response/reasoning evidence.

GenAI-Aug-HomeAssignment - S03-GenerateAPITestCasesFromSwaggerCode

Save

Share

POST

https://api.groq.com/openai/v1/chat/completions

Send

Docs

Params

Authorization

Headers 10

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Schema

Beautify

```
1 {
2   "model": "openai/gpt-oss-120b",
3   "messages": [
4     {
5       "role": "system",
6       "content": "You are a Senior API Test Automation Architect with strong expertise in Java, Rest Assured, TestNG, Maven, and API schema-driven test generation."
7     },
8     {
9       "role": "user",
10      "content": "I - INSTRUCTION\nGenerate Rest Assured Java code for Swagger Petstore APIs.\n\nMANDATORY RULES:\n1) Generate ONLY Java Rest Assured code.\n2) Do NOT include explanations, markdown, or extra text.\n3) Use TestNG as the test runner.\n4) Use meaningful class, method, and variable names.\n5) Include imports required for Rest Assured, TestNG, and assertions.\n6) Use reusable base URI setup.\n7) Cover CRUD operations for Pet resource.\n8) Add assertions for status code and important response fields.\n9) Keep the code clean, executable, and production-style.\n10) Do NOT use placeholder comments.\n\nCONTEXT\nThe code will be used by QA engineers for API automation practice using Swagger Petstore.\n\nPERSONA\nYou are a Senior API Test Automation Architect creating enterprise-style reusable Rest Assured automation code.\n\nOUTPUT\nReturn ONLY executable Java code with:\n- One test class\n- @BeforeClass setup\n- Test methods for createPet, getPetById, updatePet, deletePet\n- Request body creation\n- Response validation\n\nRules:\n- No markdown fences\n- No explanation text\n- No assumptions outside Swagger Petstore Pet API\n- Use business-readable method names\n\nTARGET\nAPI:\nSwagger Petstore\nResource: Pet\nOperations required:\n- Add a new pet\n- Find pet by ID\n- Update an existing pet\n- Delete a pet"
```

Body

Cookies 1

Headers 21

Test Results

200 OK

2.66 s

2.57 KB

Save Response

JSON

Preview

Visualize

```
1 {
2   "id": "chatcmpl-8842f356-a820-4ab7-bf8f-99c690e12ac9",
3   "object": "chat.completion",
4   "created": 1786637823,
5   "model": "openai/gpt-oss-120b",
6   "choices": [
7     {
8       "index": 0,
9       "message": {
10        "role": "assistant",
11        "content": "import io.restassured.RestAssured;\nimport io.restassured.http.ContentType;\nimport io.restassured.response.Response;\nimport org.testng.annotations.\nBeforeClass;\nimport org.testng.annotations.Test;\nimport java.util.HashMap;\nimport java.util.Map;\nimport static org.hamcrest.Matchers.*;\n\npublic class\nPetApiTests {\n\n    private static final String BASE_URI = \"https://petstore.swagger.io\";\n    private static final String BASE_PATH = \"/v2\";\n    private\n    static long petId;\n\n    @BeforeClass\n    public void setUp() {\n        RestAssured.baseURI = BASE_URI;\n        RestAssured.basePath = BASE_PATH;\n        petId\n        = System.currentTimeMillis(); // simple unique id\n    }\n\n    @Test(priority = 1)\n    public void createPet() {\n        Map<String, Object> pet = new HashMap<>\n        ();\n        pet.put(\"id\", petId);\n        pet.put(\"name\", \"Fluffy\");\n        pet.put(\"status\", \"available\");\n        pet.put(\"photoUrls\", new String\n        [] {\"example.com/photo1.jpg\"});\n        Response response = RestAssured.given()\n        .contentType(ContentType.JSON)\n        .body\n        (pet)\n        .when()\n        .post(\"/pet\")\n        .then()\n        .statusCode(200)\n        .body(\"id\", equalTo\n        ((int) petId))\n        .body(\"name\", equalTo(\"Fluffy\"))\n        .body(\"status\", equalTo(\"available\"))\n        .extract()\n        .response();\n    }\n\n    @Test(priority = 2, dependsOnMethods = {\"createPet\"})\n    public void getPetById() {\n        RestAssured.given()\n        .get(\"/pet/{id}\", petId)\n        .then()\n        .statusCode(200)\n        .body(\"id\", equalTo(petId))\n        .body(\"name\", equalTo(\"Fluffy\"))\n        .body(\"status\", equalTo(\"available\"))\n        .body(\"photoUrls\", hasSize(1))\n        .extract()\n        .response();\n    }\n\n    @Test(priority = 3, dependsOnMethods = {\"createPet\", \"getPetById\"})\n    public void updatePet() {\n        Map<String, Object> updatedPet = new HashMap<>\n        ();\n        updatedPet.put(\"id\", petId);\n        updatedPet.put(\"name\", \"Buffy\");\n        updatedPet.put(\"status\", \"pending\");\n        updatedPet.put(\"photoUrls\", new String\n        [] {\"example.com/photo2.jpg\"});\n        Response response = RestAssured.given()\n        .contentType(ContentType.JSON)\n        .body\n        (updatedPet)\n        .when()\n        .put(\"/pet/{id}\", petId)\n        .then()\n        .statusCode(200)\n        .body(\"id\", equalTo(petId))\n        .body(\"name\", equalTo(\"Buffy\"))\n        .body(\"status\", equalTo(\"pending\"))\n        .body(\"photoUrls\", hasSize(1))\n        .extract()\n        .response();\n    }\n\n    @Test(priority = 4, dependsOnMethods = {\"createPet\", \"getPetById\", \"updatePet\"})\n    public void deletePet() {\n        Response response = RestAssured.given()\n        .when()\n        .delete(\"/pet/{id}\", petId)\n        .then()\n        .statusCode(204)\n        .extract()\n        .response();\n    }\n\n    @AfterClass\n    public void tearDown() {\n        // Cleanup\n    }\n\n}
```

S03 – Generate API Test Cases from Swagger Code | Execution Proof

2 of 2

Complete screenshot supplied for the Swagger-to-test-code generation request, including request and response/reasoning evidence.

The screenshot shows a REST client interface with a POST request to `https://api.groq.com/openai/v1/chat/completions`. The request body is a JSON object with a `model` field set to `"openai/gpt-oss-120b"`. The response is a 200 OK status with a JSON body containing a `reasoning` field with a long text explanation and a `usage` field with token counts.

```
1 {
2   "model": "openai/gpt-oss-120b",
3 }
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Body Cookies Headers 21 Test Results 200 OK • 2.66 s • 2.57 KB Save Response

{ JSON Preview Visualize

```
pet()\n        .then()\n        .statusCode(200)\n        .body(\n            equalTo((int) petId))\n        .body(\n            equalTo\n            (\n                \"FluffyUpdated\")\n            ))\n        .body(\n            equalTo\n            (\n                \"status\", equalTo\n            (\n                \"pending\")\n            ))\n        }\n        @Test(priority = 4, dependsOnMethods = {updatePet})\n        public void\n            deletePet()\n            {\n                RestAssured.given()\n                .pathParam\n                (\n                    \"petId\", petId\n                )\n                .when()\n                .then()\n                .statusCode(200)\n                .body(\n                    equalTo\n                    (\n                        String.valueOf\n                        (petId)\n                    ))\n                }\n                RestAssured.given()\n                .pathParam\n                (\n                    \"petId\", petId\n                )\n                .when()\n                .get\n                (\n                    \"/pet/{petId}\"\n                )\n                .then()\n                .statusCode\n                (\n                    404\n                )\n            }\n        }\n    }\n\n    \"reasoning\": \"We need to output Java code only, no markdown. Include imports, class with @BeforeClass, test methods. Use RestAssured, TestNG. Provide base URI setup, request body creation, response validation.\\n\\nSwagger Petstore base URL is https://petstore.swagger.io/v2. We'll set baseURI and basePath.\\n\\nWe need CRUD for Pet. Fields: id (long), category, name, photoUrls, tags, status. We'll create a pet with random id.\\n\\nWe'll use static variable petId to share across methods. Use JSONObject or Map for request body.\\n\\nUse given().contentType(\"application/json\") etc.\\n\\nAdd assertions: statusCode(200) etc; response body field matches.\\n\\nWe'll need import static io.restassured.RestAssured.*; import static org.hamcrest.Matchers.*; import org.testng.annotations.*;\\n\\nLet's craft code. Ensure Maven dependencies: but not needed.\\n\\nWe'll include class PetApiTests.\\n\\nWe'll use @Test(priority=4) etc.\\n\\nMake sure to use reusable methods? Keep simple.\\n\\nOk produce final code.\"",
    \"usage\": {\n      \"queue_time\": 0.282677784,\n      \"prompt_tokens\": 396,\n      \"prompt_time\": 0.020434753,\n      \"completion_tokens\": 943,\n      \"completion_time\": 1.991609704,\n      \"total_tokens\": 1339,\n      \"total_time\": 2.012084457,\n      \"completion_tokens_details\": {\n        \"reasoning_tokens\": 213\n      }\n    }\n  },\n  \"usage_breakdown\": null,\n  \"system_fingerprint\": \"fp_c56a34c3f3\",\n  \"x_groq\": {\n    \"id\": \"req_01kzxtvsae309f1aiftzkbxbxeq\",\n    \"seed\": 170460979\n  },\n  \"service_tier\": \"on_demand\"\n}
```

Clipboard Vault Tools